

RouteRLT: Learning When and Which RL Specialist Should Control a Vision–Language–Action Policy

Chongyu Zhu¹, Jaden Hinds¹, Hyegang Kim¹, Juan Sebastian Rojas¹, Ramy Elmallah¹, Chi-Guhn Lee¹

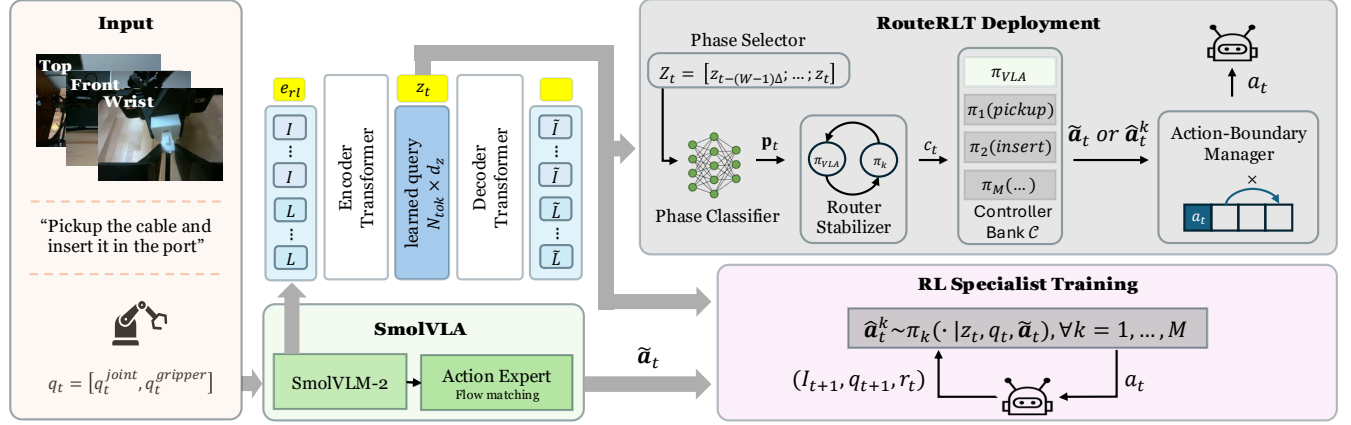


Fig. 1. Overview of RouteRLT. The frozen VLA produces a reference action chunk $\tilde{a}_t$, while the representation encoder provides the compact state z_t . A causal phase classifier predicts a controller posterior p_t ; the router stabilizer selects the active controller c_t from the controller bank $\mathcal{C} = \{\pi_{VLA}\} \cup \Pi_{RL}$, which comprises the base VLA and a bank of phase-specific RL specialists. The action-boundary manager invalidates the unexecuted chunk suffix whenever controller ownership changes and sends only the current action a_t to the robot.

Abstract— Vision–language–action (VLA) models provide broad manipulation competence, but often struggle during the precision-critical stages that dominate contact-rich industrial tasks such as connector insertion and cable management. A common remedy is to refine a pretrained VLA with reinforcement learning (RL), enabling task-specific improvement beyond behavior cloning. However, how to preserve its generalist behavior while deciding when RL refinement is needed and which specialized policy should act remains an open question. In this work, we present *RouteRLT*, a routing framework that learns when and which *RL specialist*, an RL policy trained for a single precision-critical phase, should take control from a generalist VLA. A *phase selector* identifies the active controller, a *stabilizer* suppresses transient switches, and an *action-boundary manager* handles transitions between chunked policy outputs. We evaluate RouteRLT on multi-object pick-and-place tasks in LIBERO, as well as on a real-world cable pickup and port-insertion task with multiple precision-critical stages. In simulation, the learned routing improves over the base VLA and matches routing with privileged phase boundaries, without accessing those boundaries at deployment. The real-robot evaluation validates automatic routing to both the pickup and insertion specialists under an operator-aligned handoff protocol. Altogether, these results show that learned routing applies RL specialist control where precise adaptation is most valuable while preserving generalist VLA behavior, including recovery from failed execution attempts.

I. INTRODUCTION

Pretrained vision–language–action (VLA) models compress visual observations, language instructions, and robot state

representations into general-purpose manipulation policies [1]. Broad competence, however, does not guarantee the precision and speed that contact-rich industrial tasks, such as cable management, demand [2]. For example, in a wire insertion task, a policy that reliably approaches and transports a connector may still stall or misalign during grasp closure and final insertion [3]. Importantly, in these kinds of precision-critical scenarios, the failures are not spread evenly over the trajectory, such that a small fraction of the horizon determines whether or not success is achieved in the task. This concentration of failures motivates lightweight reinforcement learning (RL) policies trained for individual precision-critical phases, which we refer to as RL specialists. Such specialization avoids spending scarce resources on segments that the generalist already performs well and reduces the risk of overwriting the long-horizon behavior that motivates using a pretrained policy. Accordingly, any practical system must decide not only how to refine behavior, but where along the trajectory refinement should take effect. Figure 1 summarizes the generalist–specialist architecture studied here.

To this end, recent work has largely converged on refining a *part* of the trajectory rather than all of it, whether by training wherever an operator intervenes [4], by restricting updates to a latent space of a frozen policy [5], or by applying a lightweight controller to one designated critical phase [6]. In all of these systems, however, the scope of refinement is set by a signal that exists only during data collection: a human operator who selects the handoff point, or, in simulation, a privileged router reading state that no robot observes at test time. To the best of our knowledge, no existing system

¹Department of Mechanical and Industrial Engineering, University of Toronto, Toronto, ON, Canada.

Corresponding author: Chongyu Zhu, cyzhu@mie.utoronto.ca

Accepted at the IROS 2026 International Workshop on Industrial Applications of Robot Learning (IARL).

predicts controller ownership over a bank of RL specialists from the generalist’s own internal state, decided at every control query, and honored mid-chunk.

In this work, we close this gap by making controller ownership a learned decision, predicted at every control query from a compact latent read out of the frozen generalist’s own internal representations [6], without any privileged signal at deployment. We make the following contributions:

- We introduce *RouteRLT*, a controller-routing framework that coordinates a frozen generalist VLA with a bank of RL specialists. A *phase selector* predicts controller ownership from the latent, a *router stabilizer* suppresses transient switches, and an *action-boundary manager* invalidates the unexecuted chunk suffix so that a change of ownership takes effect immediately rather than at the next replanning boundary.
- The framework extends to multi-object tasks and to trajectories containing multiple precision-critical stages, where ownership must be resolved among several controllers rather than toggled between two.
- Across simulation and a real-robot cable pickup and insertion task, RouteRLT raises full-task success from 85.00% to 92.22% in simulation and full-trajectory insertion success from 6.7% to 35.0% on hardware over the generalist baseline, while matching routing driven by privileged boundaries.

II. RELATED WORK

A. Vision–Language–Action Models

Modern general-purpose manipulation policies adapt web-scale multimodal pretraining to robot action prediction, whether by co-finetuning a vision–language model on robot trajectories [1], by scaling a single policy over large real-robot datasets [7], or by coupling a pretrained VLM to a continuous action expert for high-rate control [8]. Their action heads are fit by behavior cloning, however, so they cannot exceed the precision of their demonstrations and can struggle during the precision-critical stages of long-horizon tasks [3], [9]. We use SmolVLA [10], a compact backbone of this family, as the frozen generalist throughout this work, since its small footprint keeps online specialist training within a modest compute budget.

B. RL Refinement of VLA Models

Constrained by the precision limits above, a growing body of work refines a pretrained VLA with online RL rather than with additional demonstrations. These approaches differ mainly in what they update: the policy itself under operator intervention [4], [11], a latent space of a frozen policy [5], or nothing at all, re-ranking sampled actions with an offline-RL critic [12]. RLT targets a predefined critical phase by compressing internal VLA image-token representations into a compact readout token that drives a lightweight actor–critic controller [6]. This makes it suitable for phase-local adaptation without updating the generalist, but its control scope remains fixed by a signal supplied during data collection.

TABLE I

GENERALIST–SPECIALIST CONTROLLER HANDOFF IN PRIOR WORK.

Method	Trigger	Priv.	Target	Res.
HIL-SERL [4]	operator	yes	human	free
Sirius [13]	operator	yes	human	free
RLT [6]	operator	yes	RL policy	phase
SafeDagger [14]	action discrep.	no	human	step
LazyDagger [15]	action discrep.	no	human	step
EnsembleDagger [16]	ens. variance	no	human	step
ThriftyDagger [17]	novelty + risk	no	human	step
Sirius-Fleet [18]	latent anomaly	no	human	step
Sentinel [19]	chunk inconsist.	no	human	chunk
Recovery RL [20]	safety critic	no	recovery pol.	step
ASC [21]	OOD state	no	corrective pol.	step
HYDRA [22]	predicted mode	no	action space	step
SayCan [23]	lang. affordance	no	skill	subtask
SDP [24]	task index	no	frozen expert	task
AtomicVLA [25]	skill identity	no	skill expert	subtask
RouteRLT (ours)	VLA latent	no	RL specialist	query [†]

Priv. marks triggers that read privileged or operator-supplied state

unavailable at deployment; *Res.* is the temporal resolution of the decision.

[†]RouteRLT decides at each configured router query and applies an accepted switch at the next action, invalidating the unexecuted chunk suffix rather than waiting for the next replan.

C. Generalist–Specialist Control

Deciding when one controller should yield to another has been studied longest in interactive imitation learning, where the mechanisms differ in the signal they read, the target they hand control to, and the temporal resolution at which ownership changes [13]–[25]. Table I places one representative system per family on these axes; our stabilizer follows LazyDagger’s asymmetric enter and exit thresholds [15]. Multi-Expert Synthesis coordinates learned experts for locomotion and manipulation [26], while RPG focuses on smooth transitions among humanoid motion skills [27]. These works coordinate policies trained jointly as a multi-expert system, whereas we study handoffs between an existing generalist VLA and separately trained, phase-specific RL policies.

III. ROUTERLT

In this section, we present RouteRLT, a routing framework that learns when and which RL specialist should take control from a generalist VLA, in which a *phase selector* identifies the active controller, a *stabilizer* suppresses transient switches, and an *action-boundary manager* handles transitions between chunked policy outputs.

A. Problem Formulation

At control step t , the system receives synchronized visual observations I_t from multiple camera views, proprioception $q_t = [q_t^{\text{joint}}, q_t^{\text{gripper}}]$, and a language instruction ℓ . The frozen VLA encodes these inputs into its native token sequence, from which its action expert decodes an H -step proposal $\tilde{a}_t^{\text{VLA}}$. Following RLT [6], a learned token e_t is appended to the image–language prefix. Full self-attention is applied to the augmented sequence, and the hidden state at the appended position gives the compact state $z_t \in \mathbb{R}^{d_z}$. During

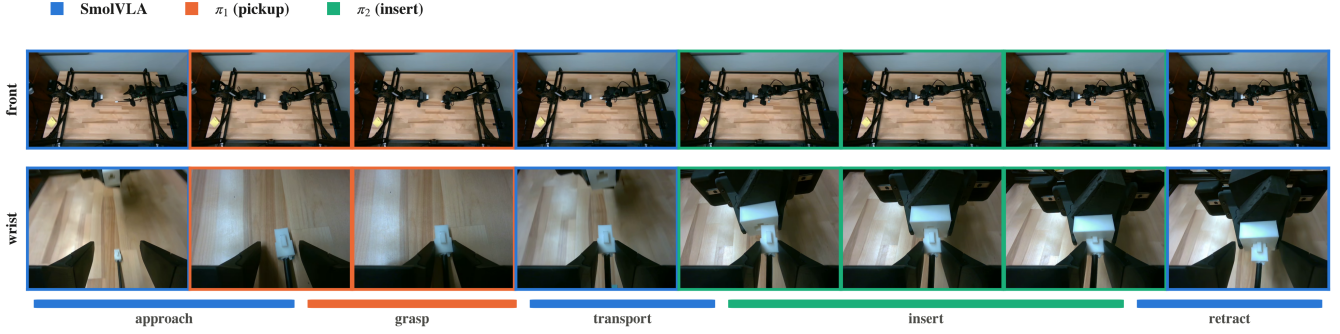


Fig. 2. Representative RouteRLT deployment on the Trossen insertion task in real time. Synchronized front and wrist views show the active-controller sequence over one rollout, with controller ownership indicated by the frame borders and lower timeline. Under the operator-aligned handoff protocol, the router selects both specialists automatically, while insertion execution begins after operator alignment.

representation learning, a parallel decoder inspired by the open-source RLinf implementation [28] reconstructs the prefix tokens from learned queries conditioned on z_t (see Fig. 1). The decoder is then discarded and the encoder is frozen. RouteRLT uses z_t as the shared perceptual input to both the phase-specific RL specialists and the phase selector; proprioception and the VLA reference action are provided only to the specialists. The routing, stabilization, and boundary-management components are independent of the actor-critic design and can support other specialist learners with compatible state and action interfaces.

The VLA and RL specialists may use different execution horizons. A VLA query queues the first $C_{\text{VLA}} \leq H$ actions of its proposal, whereas a specialist emits a C -step chunk, $C \leq H$. We write $\tilde{\mathbf{a}}_t = \tilde{\mathbf{a}}_{t,1:C}^{\text{VLA}}$ for the VLA reference supplied to a specialist and $\hat{\mathbf{a}}_t^k$ for the chunk produced by specialist k .

Let $\Pi_{\text{RL}} = \{\pi_1, \dots, \pi_M\}$ denote the RL specialist bank, where each π_k is a lightweight RL policy trained for one precision-critical phase k . The controller bank is $\mathcal{C} = \{\pi_{\text{VLA}}\} \cup \Pi_{\text{RL}}$, and $g: \mathcal{K} \rightarrow \mathcal{C}$ is the fixed phase-to-controller map: non-critical phases map to π_{VLA} , each critical phase to its specialist. The routing dataset maps each annotated phase $y_t \in \mathcal{K}$ to a controller label $\bar{y}_t = g(y_t) \in \mathcal{C}$ and pairs it with z_t . The phase selector predicts the active controller directly; its output $\mathbf{p}_t$ is a probability distribution over $\mathcal{C}$, with p_t denoting the specialist probability in the binary case. The objective is to recover the performance of privileged phase-based routing without access to phase labels or privileged task state at deployment.

B. RL Specialists

Each RL specialist emits a $C \times d_a$ action chunk, sampled from a diagonal Gaussian with fixed standard deviation and conditioned on the reference chunk produced by the base VLA:

$$\hat{\mathbf{a}}_t^k \sim \pi_k(\cdot \mid z_t, q_t, \tilde{\mathbf{a}}_t). \quad (1)$$

To prevent the actor from collapsing into a perturbation of that reference, the reference is zeroed with probability p_{drop} during training while the deviation penalty is always measured against the true reference, forcing an independent $(z_t, q_t) \mapsto \hat{\mathbf{a}}_t^k$ pathway.

Let $s_t = (z_t, q_t)$. Following RLT [6], the actor balances predicted return against deviation from the VLA proposal; we add a temporal-smoothness penalty $\mathcal{L}_\Delta$:

$$\mathcal{L}_{\text{actor}} = -\lambda_Q(n) Q_1(s_t, \hat{\mathbf{a}}_t^k) + \frac{\lambda_{\text{ref}}(n)}{C d_a} \|\hat{\mathbf{a}}_t^k - \tilde{\mathbf{a}}_t\|_2^2 + \lambda_\Delta \mathcal{L}_\Delta(\hat{\mathbf{a}}_t^k). \quad (2)$$

Here $\mathcal{L}_\Delta = [(C-1)d_a]^{-1} \sum_{j=1}^{C-1} \|\hat{\mathbf{a}}_{t,j+1}^k - \hat{\mathbf{a}}_{t,j}^k\|_2^2$ penalizes variation between consecutive actions, with strength λ_Δ . Each specialist uses twin critics. The next-state action is drawn from the current actor rather than a target actor, evaluated by Polyak-averaged twin target critics, and their minimum forms the temporal-difference target; the actor maximizes the current Q_1 alone. The return and reference coefficients may be scheduled over training step n ; decreasing λ_{ref} while increasing λ_Q anchors the specialist early and lets it depart from the reference once its critic is informative.

Specialist training is offline-to-online (see Fig. 1): an actor-critic pair is first fit on previously collected trajectories, then fine-tuned from a replay buffer that mixes those offline transitions with fresh online experience. Warm-up rollouts execute the VLA reference before the specialist takes control. Online fine-tuning uses an update-to-data ratio of 5. The VLA remains frozen throughout; the representation encoder is optimized once during representation learning and then frozen before specialist and router training.

Specialist learning is chunk-level: transitions are anchored every Δ_b control steps and cover up to the following C executed actions, allowing overlapping windows and windows that cross a replan boundary. For a window of length $L \leq C$, the target accumulates its realized discounted reward and bootstraps with γ^L ; open windows are closed at termination. Tasks use a sparse terminal success reward without shaping.

C. Latent-Space Phase Router

The router consumes only the compact states z_t and introduces no additional observation encoder. It has no access to raw observations, rewards, or privileged task state, leaving z_t as its sole input.

Algorithm 1 RouterRLT router fitting and boundary-aware deployment.

Require: Frozen VLA π_{VLA} , representation encoder E_ϕ , controller bank $\mathcal{C}$; controller-labeled splits $\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}}$; stabilizer grid $\mathcal{G}$; window length $W = 2$; query interval Δ_r ; horizons C_{VLA} and C

Router fitting

- 1: Extract z_t and form causal windows Z_t on $\mathcal{D}_{\text{tr}}$ and $\mathcal{D}_{\text{val}}$
- 2: Fit f_ϕ on $\mathcal{D}_{\text{tr}}$ using class-balanced BCE (binary) or cross-entropy (multi-controller)
- 3: Select ϕ^* by early stopping on validation AUPRC
- 4: **for all** $\eta \in \mathcal{G}$ **do**
- 5: Score stabilized validation predictions with $J_{\text{val}}(\phi^*, \eta)$
- 6: **end for**
- 7: $\eta^* \leftarrow \arg \max_{\eta \in \mathcal{G}} J_{\text{val}}(\phi^*, \eta)$ subject to the routing constraints; freeze f_{ϕ^*} and η^*

Boundary-aware deployment

- 8: Initialize $c_{-1} \leftarrow \pi_{\text{VLA}}$, stabilizer state, and empty action queue $\mathcal{Q}$
- 9: **for** $t = 0, 1, \dots$ **do**
- 10: **if** $t \bmod \Delta_r = 0$ **then**
- 11: Extract z_t , update Z_t , and predict controller posterior $\mathbf{p}_t$
- 12: $c_t \leftarrow \text{Stabilize}(\mathbf{p}_t, c_{t-1}; \eta^*) \triangleright$ Eq. 5 in the binary case
- 13: **else**
- 14: $c_t \leftarrow c_{t-1}$
- 15: **end if**
- 16: **if** $c_t \neq c_{t-1}$ **then**
- 17: Clear $\mathcal{Q}$ $\triangleright$ discard the stale chunk suffix
- 18: **end if**
- 19: **if** $\mathcal{Q}$ is empty **then**
- 20: Evaluate the VLA on the current observation to obtain z_t and $\tilde{\mathbf{a}}_t^{\text{VLA}}$
- 21: **if** $c_t = \pi_{\text{VLA}}$ **then**
- 22: $\mathcal{Q} \leftarrow \tilde{\mathbf{a}}_{t,1:C_{\text{VLA}}}^{\text{VLA}}$
- 23: **else**
- 24: $\tilde{\mathbf{a}}_t \leftarrow \tilde{\mathbf{a}}_{t,1:C}^{\text{VLA}}$; let k satisfy $c_t = \pi_k$; sample $\hat{\mathbf{a}}_t^k \sim \pi_k(\cdot | z_t, q_t, \tilde{\mathbf{a}}_t)$ and set $\mathcal{Q} \leftarrow \hat{\mathbf{a}}_t^k$
- 25: **end if**
- 26: **end if**
- 27: Execute $a_t \leftarrow \text{popfront}(\mathcal{Q})$
- 28: **end for**

The phase selector forms a causal window of W latents with stride Δ , ordered oldest first,

$$Z_t = [z_{t-(W-1)\Delta}; \dots; z_{t-\Delta}; z_t]. \quad (3)$$

Each latent is projected by a shared $\text{Linear}(d_z \rightarrow 64)$ layer with ReLU, weights tied across the W positions, and the concatenated projections are mapped to a single logit:

$$p_t = \sigma(f_\phi(Z_t)) = p_\phi(\bar{y}_t = \pi_1 | Z_t), \quad (4)$$

where $\bar{y}_t = \pi_1$ indicates that the specialist should hold control. We train f_ϕ with class-balanced binary cross-entropy, weighting the positive class by $(N - N_+)/N_+$; the multi-controller form uses class-balanced cross-entropy. The selector is evaluated every Δ_r control steps, independently of the controllers' execution horizons. Input standardization statistics are fitted on the training split over single latents and stored with the weights. We use $W = 2$ throughout and share projection weights across the two positions.

Eq. 4 is the binary form used when one specialist competes with the base VLA. For a bank of M specialists, f_ϕ instead produces a $|\mathcal{C}|$ -way softmax, with $[\mathbf{p}_t]_c = p_\phi(\bar{y}_t = c | Z_t)$ for $c \in \mathcal{C}$.

The stabilizer that turns p_t into a controller is a three-phase state machine. Let $\psi_t \in \{\text{PRE}, \text{ACTIVE}, \text{POST}\}$, where PRE and POST select the base VLA and ACTIVE selects the specialist, thereby determining c_t ; the two VLA phases differ only in whether the specialist has already held control. Let n_t^+ count the consecutive steps ending at t with $p_t \geq \tau_{\text{enter}}$, let n_t^- count those with $p_t < \tau_{\text{exit}}$, and let m_t be the number of steps since the most recent entry. The entry guard E_t^+ holds when $\psi_{t-1} \neq \text{ACTIVE}$ and $n_t^+ \geq d_{\text{enter}}$. The exit guard E_t^- holds when $\psi_{t-1} = \text{ACTIVE}$, $n_t^- \geq d_{\text{exit}}$, and $m_t \geq d_{\text{occ}}$. The transition rule is then

$$\psi_t = \begin{cases} \text{ACTIVE}, & \text{if } E_t^+, \\ \text{POST}, & \text{if } E_t^-, \\ \psi_{t-1}, & \text{otherwise,} \end{cases} \quad (5)$$

with $\eta = (\tau_{\text{enter}}, \tau_{\text{exit}}, d_{\text{enter}}, d_{\text{exit}}, d_{\text{occ}})$. The dwell counters count consecutive *confirming* decisions rather than elapsed time, and the occupancy minimum d_{occ} is asymmetric: it constrains exits only and never delays an entry. Together, hysteresis and dwell suppress transient switches. For a bank of M specialists, the same guards operate on the corresponding entries of $\mathbf{p}_t$, and the entry rule admits the highest-confidence eligible controller.

Selection is two-level, and both levels read validation data, so validation quantities are selection statistics rather than held-out ones. Classifier weights are chosen by early stopping on validation AUPRC. The stabilizer parameters η are then chosen by grid search on the validation criterion, subject to a floor on entry recall and a ceiling on false entries per episode, tie-broken by fewer switches. The architecture and window length are fixed across seeds; weights and η are fit per seed, before any test episode is run.

D. Boundary-Aware Chunk Execution

Both the VLA and RL specialists emit action chunks, so a controller switch can fall in the middle of a chunk. The action-boundary manager (see Fig. 1) owns this transition. When c_t changes, it invalidates the unexecuted suffix and queries the incoming controller immediately. A handoff to the VLA queues the first C_{VLA} actions of a fresh proposal. A handoff to specialist π_k obtains a fresh VLA proposal from the same observation, uses its first C actions as $\tilde{\mathbf{a}}_t$, and samples $\hat{\mathbf{a}}_t^k$. The environment executes only the front action $a_t = \text{popfront}(\mathcal{Q})$. The replay buffer used for specialist training stores only executed actions, together with the selected controller and next compact state z_{t+1} . This keeps action ownership and learning targets consistent across controller transitions; every evaluation episode is verified to contain no queued action from a superseded plan or inactive controller.

E. Training Protocol

Training proceeds in four stages. First, the base VLA is fine-tuned on task demonstrations. Second, the representation

encoder is trained to reconstruct the frozen VLA prefix and then frozen to define z_t . Third, each RL specialist is fit offline on previously collected trajectories and then fine-tuned online under the privileged phase boundary for its target phase, sampling throughout from a replay buffer that mixes offline and online transitions. Fourth, the specialists are frozen and used to collect labeled routing data under privileged routing. Algorithm 1 summarizes router fitting and boundary-aware deployment; representation and specialist learning build on RLT [6] through the components described above.

IV. EXPERIMENTS

Our evaluation comprises two complementary settings: multi-object pick-and-place tasks from LIBERO Object in simulation [29], and cable pickup and port insertion on a Trossen Stationary AI platform derived from ALOHA [30], [31]. We present the primary comparisons for each setting together with targeted ablations of the representation inputs inherited from RLT [6] and control scope. Figure 3 summarizes both evaluation settings and the 1 mm-clearance port used on hardware. Unless otherwise noted, simulation success rates are reported as mean $\pm$ sample standard deviation over three independently trained seeds.

A. LIBERO Simulation

We use LIBERO Object [29] to test two questions: whether an RL specialist improves the precision-critical pickup phase, and whether learned routing can retain this benefit without privileged phase information. The evaluation spans tomato sauce, butter, and chocolate pudding tasks under identical initial conditions and episode ordering across methods.

a) Evaluation protocol: All methods use the same 60 held-out episodes, task order, and 300-step horizon at 20 Hz as $C_{VLA} = 10$ and $C = 5$. In the direct routing comparison, Oracle-routed RLT and RouteRLT also share the VLA, representation encoder, specialist checkpoint, controller horizons, and episode initializations; only the source of controller transitions differs, with external phase boundaries for Oracle-routed RLT and predictions from z_t for RouteRLT. The pickup and full-trajectory specialists each receive 200 online-training episodes, with $\lambda_{\Delta} = 0$. We set $\Delta_r = 1$, querying the router at every environment step; its $W = 2$ window uses the current z_t and the state encoded 0.5 s earlier. Table II reports full-task success as the primary metric, together with pickup success and successes per 1,000 environment steps to measure critical-stage performance and interaction efficiency.

b) Results: Table II supports three conclusions. First, neither more frequent planning nor privileged self-routing improves the base VLA: SmolVLA@5 is weaker than the default SmolVLA@10, and Oracle self-route remains at baseline performance. Directly replanning SmolVLA therefore does not resolve its pickup failures. Second, under the same privileged boundary, Oracle-routed RLT raises full-task success from 83.33% to 91.11%. This controlled comparison confirms the value of the phase-specific RL specialist conditioned on z_t and the VLA proposal. Third,

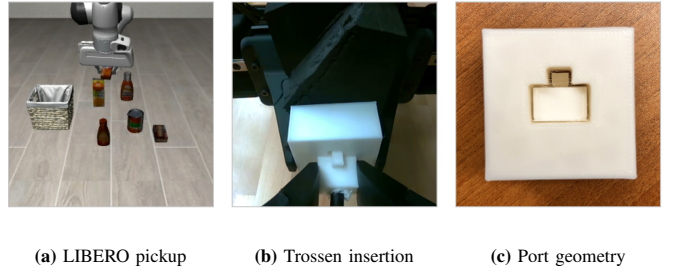


Fig. 3. Evaluation tasks and representative observations: (a) a LIBERO pickup after lift-off; (b) a successful Trossen insertion viewed from the wrist camera; and (c) the 3D-printed port used for hardware evaluation, with 1 mm clearance.

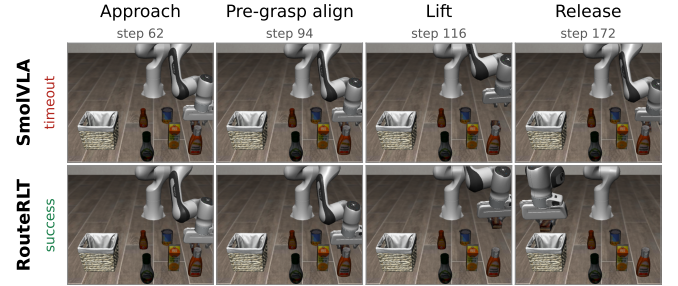


Fig. 4. Representative paired rollout on a held-out LIBERO task under identical initial conditions for the instruction “pick up the chocolate pudding and place it in the basket.” At matched environment steps, the SmolVLA@10 baseline (top) closes without securing the object and reaches the 300-step limit, whereas RouteRLT (bottom) aligns, lifts, and releases the pudding over the basket, completing the task at step 189.

RouteRLT reaches 92.22%, retaining the specialist’s benefit while achieving the highest successes per 1,000 steps. Learned routing therefore recovers the performance of privileged routing without paying for the gain through longer trajectories.

The relationship between pickup and full-task success further localizes this effect. SmolVLA@5 improves pickup over SmolVLA@10 but is weaker on full-task success, showing that more frequent planning can clear the intermediate metric without improving the complete trajectory. In contrast, the pickup-to-full-task gaps of Oracle-routed RLT and RouteRLT are only 1.11 and 0.56 percentage points, respectively. Once their pickup specialist succeeds, the VLA therefore usually completes the remaining transport and placement phases. This supports the central control allocation in RouteRLT: specialize the precision-critical phase while preserving the generalist for phases it already handles reliably. Moreover, RouteRLT improves successes per 1,000 steps by 14.3% relative to SmolVLA@10, so its higher task success also translates into more completed tasks under a fixed interaction budget.

Figure 4 shows a matched held-out case in which the learned handoff enables successful pickup and placement from an initial condition where SmolVLA@10 fails to secure the object.

c) Routing analysis: The task-level result is explained by reliable coverage of the critical phase: the router reaches every target pickup interval with entry recall 1.0 and zero-step median delay, while 2.12 controller switches per episode

TABLE II

LIBERO MULTI-OBJECT PICK-AND-PLACE PERFORMANCE. PRIMARY COMPARISONS EVALUATE LEARNED ROUTING AGAINST SMOLVLA AND PRIVILEGED-BOUNDARY REFERENCES; ABLATIONS EVALUATE REPRESENTATION INPUTS AND CONTROL SCOPE.

Method	Full-task success (%)	Pickup success (%)	Successes / 1k steps
<i>Primary comparisons</i>			
SmolVLA@5	82.50 ± 5.90	90.00 ± 0.00	4.65 ± 0.47
SmolVLA@10	85.00 ± 0.00	85.00 ± 0.00	4.97 ± 0.03
Oracle self-route	83.33 ± 0.00	83.33 ± 0.00	4.80 ± 0.00
Oracle-routed RLT	91.11 ± 0.96	92.22 ± 0.96	5.56 ± 0.09
RouteRLT (image + language)	92.22 ± 2.55	92.78 ± 1.92	5.68 ± 0.24
<i>Ablations</i>			
RouteRLT (image only)	90.56 ± 1.93	92.22 ± 0.96	5.48 ± 0.17
Full-trajectory RL specialist	88.89 ± 1.92	90.56 ± 0.96	5.37 ± 0.13

SmolVLA@ K replans every K environment steps; @10 is the default cadence, while @5 matches the five-step control cadence of the RL specialist. Oracle self-route retains SmolVLA control within privileged pickup boundaries, whereas Oracle-routed RLT invokes the pickup specialist within the same boundaries. RouteRLT (image + language) is the proposed method. RouteRLT (image only) removes language tokens from the inputs used to construct z_t . Full-trajectory RL specialist removes the router and trains one specialist over all phases on the full-task reward under the same interaction and update budgets, retaining z_t and the VLA proposal as inputs.

remain close to the ideal two-switch sequence. A strict boundary comparison initially counts near-boundary timing offsets as false entries: 49 of 180 episodes enter one or two steps before the privileged boundary, yet 47 of these episodes succeed. Only eight episodes contain a genuinely spurious entry or re-entry, and five still complete the task. Thus an off-phase activation is not itself a failure: timely coverage of the true pickup interval is more important, while the stabilizer can return control and the action-boundary manager discards the stale specialist suffix. Instead, 10 of the 14 failures occur because pickup never completes, so neither the learned nor privileged router observes an exit. The remaining spurious transitions likely reflect ambiguous observations near phase changes and the state-distribution shift between privileged-routed training trajectories and learned-router deployment; collecting routing labels under the deployed policy is a natural next step.

d) Ablation: Representation input. The ablation rows in Table II compare matched image-only and image–language inputs used to construct z_t . The image-only variant preserves most of the pickup success but is weaker in full-task success and interaction efficiency. This separation is informative: visual cues are largely sufficient to identify when pickup occurs, but they do not reliably preserve which object the instruction selects. Although the frozen VLA’s image features already contain some multimodal context, language information retained only indirectly is insufficient for an object-conditioned compact state across the three tasks. Including language tokens directly therefore improves object identity in z_t without changing the downstream specialist or router architecture.

Control scope. To isolate control scope, we remove the router and train one lightweight RL specialist over all phases with the same interaction and update budgets as RouteRLT, retaining z_t and the SmolVLA proposal as inputs. The resulting policy improves over both SmolVLA baselines, confirming that the compact state and VLA proposal support useful online adaptation beyond the base policy. However, it remains

weaker than RouteRLT in full-task success, pickup success, and interaction efficiency. A single lightweight specialist must distribute its capacity and sparse full-task reward across phases with different control requirements, including phases where the pretrained VLA is already strong. Routing instead preserves that generalist behavior and concentrates the RL updates on pickup. Together, the two ablations show that the gain depends on both an instruction-aware compact state and phase-localized specialist control, rather than merely adding an RL policy to the full trajectory.

B. Trossen Real-Robot Insertion

We use cable pickup and port insertion to test whether multiple RL specialists improve distinct precision-critical stages and whether learned routing can compose them across one physical trajectory.

a) Evaluation protocol: The pickup specialist π_1 controls grasping, the insertion specialist π_2 controls final alignment and seating, and SmolVLA controls approach, transport, and retraction. We compare SmolVLA, Oracle-routed RLT, and RouteRLT under matched initial conditions and controller checkpoints. The oracle uses a scripted end-effector-height rule, whereas RouteRLT selects both specialists from z_t . Under the operator-aligned handoff protocol, insertion execution begins after operator alignment, although specialist selection remains automatic. We set $\Delta_r = 5$, querying the router every five control steps. We report pickup-to-fixture and final-insertion success. We use $\lambda_\Delta = 25$ for π_1 and $\lambda_\Delta = 0$ for π_2 . For full-trajectory evaluation, we use $C_{VLA} = 35$ and $C = 10$ for the VLA and specialists, respectively; the VLA horizon preserves the default simulation replanning period in wall-clock time after accounting for the hardware control rate.

b) End-to-end results: Figure 6 reveals three stage-wise effects. First, SmolVLA delivers the connector to the fixture in 53.3% of trials but seats it in only 6.7%, identifying contact-rich insertion as the dominant downstream bottleneck. Second, routed specialization raises final insertion to 33.3%

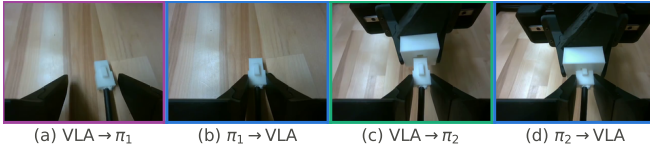


Fig. 5. Representative wrist-camera observations from a held-out router-validation trajectory at annotated controller boundaries: (a) the VLA hands control to the pickup specialist π_1 ; (b) π_1 returns control to the VLA for transport; (c) the VLA activates the insertion specialist π_2 ; and (d) π_2 returns control to the VLA after seating. These images illustrate the annotated routing phases.

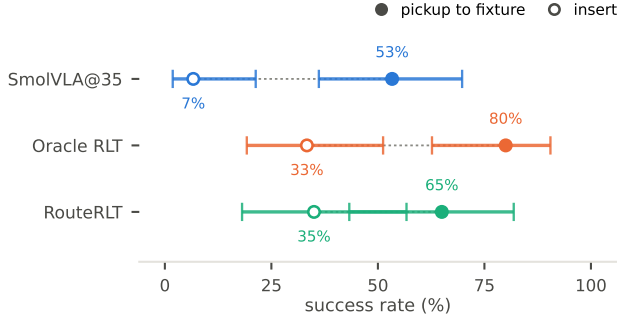


Fig. 6. Real-robot full-trajectory stage survival. Points show pickup-to-fixture and final-insertion success rates, and whiskers denote Wilson 95% confidence intervals. SmolVLA and Oracle-routed RLT use 30 trials; RouteRLT uses the same 20 trials collected under the locked insertion-exit protocol for both metrics.

with Oracle-routed RLT and 35.0% with RouteRLT, a more than fivefold gain over SmolVLA. Learned routing therefore attains final performance comparable to the manually specified height rule without using its privileged geometric trigger at deployment. Third, the oracle reaches the fixture more often than RouteRLT (80.0% versus 65.0%), yet the two attain similar final insertion. RouteRLT converts 53.8% of its fixture arrivals into seating, compared with 41.7% for the oracle. Arrival rate alone therefore does not determine the composed outcome: the state at which π_2 takes control must also match the specialist’s phase-entry distribution. Because the height rule responds only to end-effector geometry while the learned router conditions on z_t , this difference is consistent with more semantically aligned insertion handoffs. Longer pickup rollouts also exhibit retry behavior: when the connector is not secured, the system remains in the pickup portion of the task and the learned router can activate π_1 on a later decision rather than advancing to insertion.

c) Specialist capability and composition gap: Figure 7 evaluates each controller from a standardized phase-entry state with the router bypassed. The pickup specialist reaches 83.3% success compared with 0.0% for SmolVLA@10, while the insertion specialist reaches 62.1% compared with 8.0%. These isolated gaps establish that both specialists improve their designated critical stages, rather than the end-to-end gain arising from routing alone. The isolated and composed results also separate specialist capability from system composition. The isolated pickup result shows that π_1 can reliably acquire

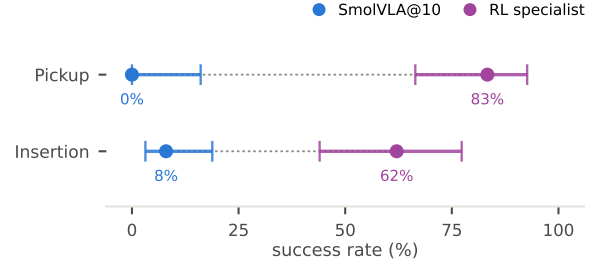


Fig. 7. Isolated real-robot success rates from standardized phase-entry states with the router bypassed. Points show controller success rates, and whiskers denote Wilson 95% confidence intervals. SmolVLA@10 replans every ten environment steps; the RL-specialist results correspond to π_1 for pickup and π_2 for insertion.

the connector from a standardized entry state; full execution then delivers it to the fixture in 65.0% of trials after adding approach, routing, and transport. Conditional seating after fixture arrival remains 53.8%, close to the isolated insertion rate of 62.1%. Full-trajectory performance is not the product of the isolated rates because those evaluations begin from standardized phase-entry states, while composed execution presents each specialist with states produced by the preceding controller. The remaining gap therefore lies primarily in upstream transport and handoff-state consistency, rather than in the absence of a capable pickup or insertion specialist.

V. LIMITATIONS

This study validates modular routing rather than general-purpose manipulation. It uses SmolVLA as the backbone and does not compare against full-backbone online fine-tuning; stronger VLAs or full-model adaptation may change the relative benefit of specialization. The LIBERO setting is near saturation, limiting its ability to distinguish routing variants. Across the two domains, we evaluate one RL specialist in simulation and two on hardware; routing over larger controller banks with less distinct phase boundaries remains untested. Hardware evaluation is restricted to one robot, connector, and port geometry, with grasp pose remaining an uncontrolled source of insertion variation; the operator-aligned handoff also does not establish fully autonomous multi-specialist composition. Finally, router and specialist learning remain decoupled. Routing labels are collected under privileged boundaries, while specialists are trained within privileged or standardized phase windows, exposing both components to out-of-distribution states during end-to-end execution.

VI. CONCLUSION

In this work, we introduced *RouteRLT*, a generalist-specialist routing framework that predicts controller ownership from frozen VLA representations, stabilizes phase transitions, and applies ownership changes immediately across chunked policy outputs. On multi-object LIBERO tasks, learned routing preserves the gain from localized RL control and matches routing based on privileged phase boundaries. On hardware, RouteRLT automatically selects the pickup and

insertion specialists across multiple precision-critical stages, substantially outperforming SmolVLA and achieving final insertion performance comparable to Oracle-routed RLT, which uses a hand-designed end-effector-height rule, under the operator-aligned handoff protocol. These results support learned controller routing as a practical way to retain generalist behavior outside critical phases while applying specialized control where additional precision is needed. Future work will evaluate stronger VLA backbones and broader hardware geometries, and jointly adapt the router and specialists to reduce handoff distribution shift and enable fully autonomous multi-specialist composition.

REFERENCES

- [1] B. Zitkovich *et al.*, “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 2165–2183.
- [2] C. Zhu, R. ElMallah, H. Kim, Z. Tang, J. Rao, A. Arutyunov, S. Ha, and C.-G. Lee, “Wirecraft: A simulation benchmark for industrial dlo manipulation,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.18097>
- [3] C. Xu, Q. Li, J. Luo, and S. Levine, “RLDG: Robotic Generalist Policy Distillation via Reinforcement Learning,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [4] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.
- [5] A. Wagenmaker *et al.*, “Steering Your Diffusion Policy with Latent Space Reinforcement Learning,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 258–282.
- [6] C. Xu *et al.*, “RL Token: Bootstrapping Online RL with Vision-Language-Action Models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [7] M. J. Kim *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 2679–2713.
- [8] K. Black *et al.*, “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [9] Y. Liu, X. Zhang, L. Liu, and A. Boularias, “Foresight residual rl for long-horizon robot manipulation with vision-language-action models,” 2026, accepted at IROS 2026. [Online]. Available: <https://arxiv.org/abs/2607.16506>
- [10] M. Shukor *et al.*, “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [11] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “ConRFT: A Reinforced Fine-Tuning Method for VLA Models via Consistency Policy,” in *Proceedings of Robotics: Science and Systems*, Jun. 2025.
- [12] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, “Steering Your Generalists: Improving Robotic Foundation Models via Value Guidance,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 4996–5013.
- [13] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, “Robot Learning on the Job: Human-in-the-Loop Autonomy and Learning During Deployment,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1727–1742, 2025.
- [14] J. Zhang and K. Cho, “Query-Efficient Imitation Learning for End-to-End Simulated Driving,” in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, ser. AAAI’17. AAAI Press, 2017, p. 2891–2897.
- [15] R. Hoque *et al.*, “LazyDagger: Reducing Context Switching in Interactive Imitation Learning,” in *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*. IEEE Press, 2021, p. 502–509.
- [16] K. Menda, K. Driggs-Campbell, and M. J. Kochenderfer, “EnsembleDagger: A Bayesian Approach to Safe Imitation Learning,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 5041–5048.
- [17] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg, “ThriftyDagger: Budget-Aware Novelty and Risk Gating for Interactive Imitation Learning,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, A. Faust, D. Hsu, and G. Neumann, Eds., vol. 164. PMLR, 08–11 Nov 2022, pp. 598–608.
- [18] H. Liu *et al.*, “Multi-Task Interactive Robot Fleet Learning with Visual World Models,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 4286–4313.
- [19] C. Agia *et al.*, “Unpacking Failure Modes of Generative Policies: Runtime Monitoring of Consistency and Progress,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 689–723.
- [20] B. Thananjeyan *et al.*, “Recovery RL: Safe Reinforcement Learning With Learned Recovery Zones,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.
- [21] N. Yokoyama *et al.*, “ASC: Adaptive Skill Coordination for Robotic Mobile Manipulation,” *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 779–786, 2024.
- [22] S. Belkhale, Y. Cui, and D. Sadigh, “HYDRA: Hybrid Robot Actions for Imitation Learning,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 2113–2133.
- [23] B. Ichter *et al.*, “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205. PMLR, 14–18 Dec 2023, pp. 287–318.
- [24] Y. Wang *et al.*, “Sparse Diffusion Policy: A Sparse, Reusable, and Flexible Policy for Robot Learning,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 649–665.
- [25] L. Zhang *et al.*, “AtomicVLA: Unlocking the Potential of Atomic Skill Learning in Robots,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 20 743–20 754.
- [26] K. Yuan and Z. Li, “Multi-expert synthesis for versatile locomotion and manipulation skills,” *Frontiers in Robotics and AI*, vol. 9, p. 970890, 2022.
- [27] Y. Xin, J. Bao, Y. Dong, X. Wang, B. Zhao, X. Li, J. Tan, and D. Wang, “RPG: Robust policy gating for smooth multi-skill transitions in humanoid fighting,” *CoRR*, vol. abs/2604.21355, 2026.
- [28] RLinf Team, “RLinf: Reinforcement learning infrastructure for embodied and agentic ai,” GitHub repository, 2026. [Online]. Available: <https://github.com/RLinf/RLinf>
- [29] B. Liu *et al.*, “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [30] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [31] Trossen Robotics, “Stationary AI: Bimanual Robotic Machine Learning Workstation,” <https://www.trossenrobotics.com/stationary-ai>, 2026, accessed: 2026-08-13.